

SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE CODE: CSA14

COURSE NAME: COMPILER DESIGN

COURSE OUTCOME COVERED

CO3: *Analyze syntax-directed translation method using synthesized and inherited attributes to generate a Parser Tree. (BL4)*

Assessment Tool Weightage: 40%

QUESTIONS:5

Duration :60 Mins
On Class
Total Marks:50

Annexure A

EXPERIMENTS

Code of Conduct:

*I Ranjitha. R (192421285) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the student:

A rectangular box containing a handwritten signature in blue ink that reads "Ranji".

Annexure A

EXPERIMENTS

Q1. Write a C program to implement a syntax-directed definition for evaluating arithmetic expressions using the following grammar:

$$E \rightarrow E + T \mid T$$
$$T \rightarrow T * F \mid F$$
$$F \rightarrow \text{num}$$

Input:

3 + 5 * 2

The program should display the attribute values and the final value of the expression.

Aim

To write a C program to implement a syntax-directed definition for evaluating an arithmetic expression using the grammar $E \rightarrow E + T \mid T$, $T \rightarrow T * F \mid F$, and $F \rightarrow \text{num}$.

Algorithm

1. Start the program.
2. Read the arithmetic expression.
3. Identify the numbers and operators.
4. Evaluate multiplication first according to the grammar.
5. Evaluate addition using the result of multiplication.
6. Display the attribute values.
7. Display the final value.
8. Stop.

Procedure

1. Initialize the values 3, 5, and 2.
2. Calculate $T = 5 * 2$.
3. Calculate $E = 3 + T$.
4. Display the intermediate attribute values.
5. Display the final result.

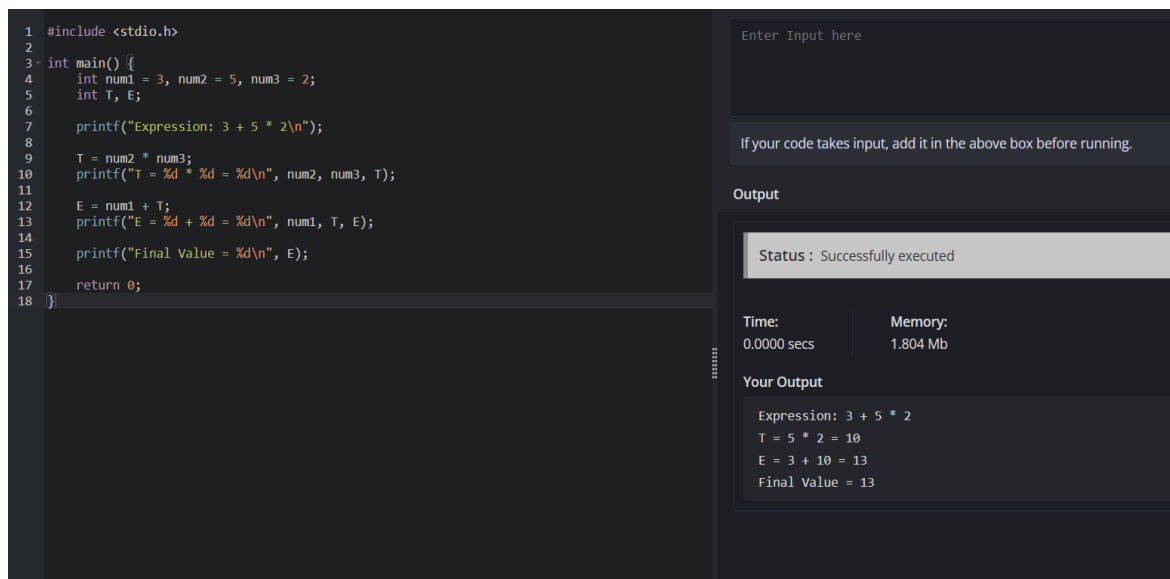
Code

```
#include <stdio.h>

int main() {
    int num1 = 3, num2 = 5, num3 = 2;
    int T, E;
    printf("Expression: 3 + 5 * 2\n");
    T = num2 * num3;
    printf("T = %d * %d = %d\n", num2, num3, T);
    E = num1 + T;
    printf("E = %d + %d = %d\n", num1, T, E);

    printf("Final Value = %d\n", E);
    return 0;
}
```

Output



The screenshot shows a C program being executed in an IDE. The code on the left is the same as the one in the 'Code' section. The right panel shows the execution results:

- Enter Input here:** (Empty text box)
- If your code takes input, add it in the above box before running.** (Instructional text)
- Status:** Successfully executed
- Time:** 0.0000 secs
- Memory:** 1.804 Mb
- Your Output:**
Expression: 3 + 5 * 2
T = 5 * 2 = 10
E = 3 + 10 = 13
Final Value = 13

Q2. Write a C program to implement a simple type checker for arithmetic expressions using the following grammar:

$E \rightarrow E + T \mid T$

$T \rightarrow T * F \mid F$

$F \rightarrow \text{id}$

Consider the following symbol table:

a : int

b : int

c : float

Input:

$a + b * c$

Determine the type of each subexpression and the final expression.

Aim

To write a C program to implement a simple type checker for arithmetic expressions using a symbol table.

Algorithm

1. Start the program.
2. Create the symbol table with identifiers and their types.
3. Read the expression $a + b * c$.
4. Find the type of each identifier.
5. Determine the type of $b * c$.
6. Determine the type of $a + (b * c)$.
7. Display the type of each subexpression.
8. Stop.

Procedure

1. Store a as int, b as int, and c as float.
2. Check the types of the identifiers.
3. Since b is int and c is float, $b * c$ gives float.
4. Since a is int and the result of $b * c$ is float, the final expression is float.
5. Display all types.

Code

```
#include <stdio.h>

int main() {
    printf("Symbol Table:\n");
    printf("a : int\n");
    printf("b : int\n");
    printf("c : float\n");
    printf("b * c : float\n");
    printf("a + (b * c) : float\n");
    printf("Final Expression Type : float\n");
    return 0;
}
```

}

Output

The screenshot shows a C program being executed in an IDE. The source code on the left is as follows:

```
1 #include <stdio.h>
2
3 int main() {
4     printf("Symbol Table:\n");
5     printf("a : int\n");
6     printf("b : int\n");
7     printf("c : float\n");
8
9     printf("b * c : float\n");
10    printf("a + (b * c) : float\n");
11    printf("Final Expression Type : float\n");
12    return 0;
13 }
14
```

On the right, the execution interface shows an empty "Enter Input here" field. Below it, a message states: "If your code takes input, add it in the above box before running." The "Output" section displays the following information:

- Status : Successfully executed
- Time: 0.0000 secs
- Memory: 1.488 Mb
- Your Output:
 - Symbol Table:
 - a : int
 - b : int
 - c : float
 - b * c : float
 - a + (b * c) : float
 - Final Expression Type : float

Q3. Write a C program to determine whether two type expressions are equivalent. Consider the following type expression

T1 = array(10, int)

T2 = array(10, int)

The program should display:

Equivalent or Not Equivalent .

Aim

To write a C program to determine whether two given type expressions are equivalent.

Algorithm

1. Start the program.
2. Store the first type expression.
3. Store the second type expression.
4. Compare both type expressions.
5. If they are equal, display Equivalent.
6. Otherwise, display Not Equivalent.
7. Stop.

Procedure

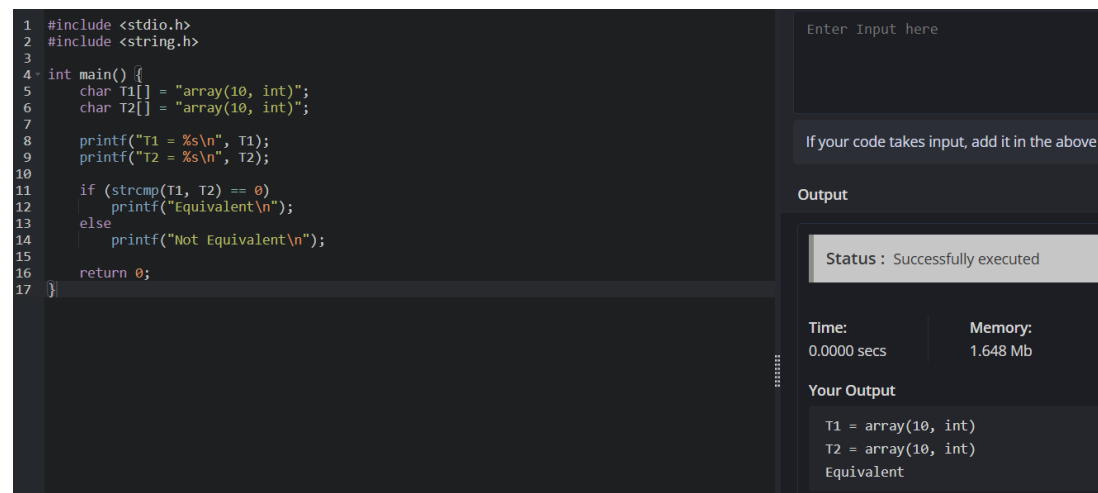
1. Store T1 = array(10, int).
2. Store T2 = array(10, int).

3. Compare T1 and T2 using strcmp().
4. Since both expressions are the same, display Equivalent.

Code

```
#include <stdio.h>
#include <string.h>
int main() {
    char T1[] = "array(10, int)";
    char T2[] = "array(10, int)";
    printf("T1 = %s\n", T1);
    printf("T2 = %s\n", T2);
    if (strcmp(T1, T2) == 0)
        printf("Equivalent\n");
    else
        printf("Not Equivalent\n");
    return 0;
}
```

Output

A screenshot of a C program being executed in an IDE. The code on the left defines two character arrays, T1 and T2, both containing the string "array(10, int)". It then uses printf to display each string and strcmp to compare them. Since the strings are identical, the program prints "Equivalent". The right side of the screenshot shows the execution environment with a status bar indicating "Successfully executed", execution time of 0.0000 secs, and memory usage of 1.648 Mb. The "Your Output" section displays the printed strings and the final comparison result.

```
1 #include <stdio.h>
2 #include <string.h>
3
4 int main() {
5     char T1[] = "array(10, int)";
6     char T2[] = "array(10, int)";
7
8     printf("T1 = %s\n", T1);
9     printf("T2 = %s\n", T2);
10
11     if (strcmp(T1, T2) == 0)
12         printf("Equivalent\n");
13     else
14         printf("Not Equivalent\n");
15
16     return 0;
17 }
```

Enter Input here

If your code takes input, add it in the above box

Output

Status : Successfully executed

Time: 0.0000 secs Memory: 1.648 Mb

Your Output

T1 = array(10, int)
T2 = array(10, int)
Equivalent

Q4. Write a C program to construct and display the syntax tree for the following expression using the given grammar:

$E \rightarrow E + T \mid T$

$T \rightarrow T * F \mid F$

$F \rightarrow \text{id}$

Input:

$a + b * c$

Display the constructed syntax tree.

Aim

To write a C program to construct and display the syntax tree for the expression $a + b * c$.

Algorithm

1. Start the program.
2. Read the expression $a + b * c$.
3. Identify $+$ as the root operator.
4. Place a as the left child.
5. Place $*$ as the right child of $+$.
6. Place b and c as children of $*$.
7. Display the syntax tree.
8. Stop.

Procedure

1. The given expression is $a + b * c$.
2. Multiplication has higher precedence than addition.
3. Therefore, the expression becomes $a + (b * c)$.
4. Create $+$ as the root.
5. Create a as its left child.
6. Create $*$ as its right child.
7. Create b and c as children of $*$.
8. Display the tree.

Code

```
#include <stdio.h>

int main() {

    printf("Syntax Tree:\n");

    printf("    +\n");

    printf("    /\n");
```

```

printf("    a *\n");

printf("    /\n");

printf("    b c\n");

return 0;

}

```

Output

The screenshot shows a C program in a code editor on the left and its execution output on the right. The code defines a main function that prints a syntax tree structure. The output panel shows the status 'Successfully executed', execution time '0.0000 secs', and memory usage '1.46 Mb'. The 'Your Output' section displays the printed syntax tree:

```

Syntax Tree:
      +
     /\
    a  *
   /\
  b  c

```

Q5. Write a C program to demonstrate the propagation of inherited and synthesized attributes using the following grammar:

$D \rightarrow T L$

$T \rightarrow \text{int}$

$L \rightarrow \text{id } L'$

$L' \rightarrow , \text{id } L' \mid \epsilon$

Input:

int a,b,c

Display the type assigned to each identifier.

Aim

To write a C program to demonstrate the propagation of inherited and synthesized attributes for assigning types to identifiers.

Algorithm

1. Start the program.
2. Read the declaration int a,b,c.
3. Identify the type as int.

4. Propagate the type to the identifier list.
5. Assign int to a, b, and c.
6. Display the type of each identifier.
7. Stop.

Procedure

1. The declaration is int a,b,c.
2. From $T \rightarrow \text{int}$, the type is obtained as int.
3. The type is propagated to the identifier list.
4. Assign int to a.
5. Assign int to b.
6. Assign int to c.
7. Display the results.

Code

```
#include <stdio.h>

int main() {
    printf("Declaration: int a,b,c\n\n");
    printf("a : int\n");
    printf("b : int\n");
    printf("c : int\n");
    return 0;
}
```

Output

The screenshot displays a C program in a code editor on the left and its execution output on the right. The code includes `<stdio.h>` and defines a `main` function that prints the declaration and type of three variables: `a`, `b`, and `c`. The output on the right shows the program was successfully executed, with a status message and execution details (Time: 0.0000 secs, Memory: 1.516 Mb). The 'Your Output' section shows the following text:

```
Declaration: int a,b,c
a : int
b : int
c : int
```

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Concepts	25	Demonstrates thorough understanding and effectively integrates multiple concepts/technologies	Good understanding with proper integration of most concepts	Basic understanding; limited or partial integration	Poor understanding; unable to integrate concepts
Experimental Design	30	Well-planned, systematic implementation with optimal solution	Correct implementation with minor issues	Basic implementation with errors or inefficiencies	Incorrect or incomplete implementation
Use of Tools	20	Effective and advanced use of tools/technologies with proper configuration	Appropriate use of tools with correct execution	Limited or inconsistent use of tools	Incorrect or minimal use of tools
Analysis of Results	15	Insightful analysis with accurate interpretation and validation of results	Good analysis with reasonable interpretation	Basic analysis with limited insights	Poor or incorrect analysis of results
Documentation	10	Clear, well-structured documentation with diagrams, code, and results	Organized documentation with necessary details	Basic documentation with missing details	Poor or incomplete documentation